



Universidad autónoma de baja california

Ingeniería en computacion

Internet de las cosas

Lab-taller 13: prototipo IoT trama binaria

Maestro: Dr. Aguilar Noriega Leocundo

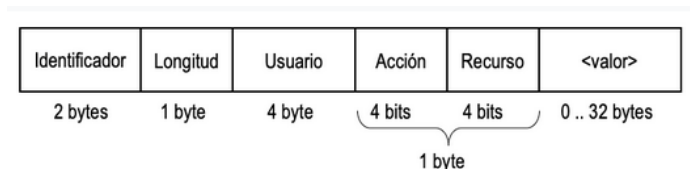
Erik Garcia Chávez 01275863

21 de abril del 2026

## Instrucciones:

Desarrollo de un prototipo de IoT con comunicación TCP entre una esp32 y un servidor,

mediante una trama binaria, la cual tendrá el siguiente formato:



En donde los campos se llenarán como:

**<Identificador>**: Es un campo que funciona inicio del frame y corresponde al valor 0xCA 0xFE.

**<Longitud>**: Un campo que contiene la longitud en bytes del resto de la trama que incluye Usuario, Acción, Recurso y valor si es que la trama lo contiene.

**<Usuario>**: Es un campo que funciona para identificar al usuario y corresponde a un valor de 4 bytes (32 bits) en el que se usa el valor de la matricula.

**<Acción>**: Un campo de 4 bits para indicar la acción (operación) que se hará sobre un determinado recurso y puede contener descritos en la Tabla 1.

**<Recurso>**: Es un campo que funciona para identificar el recurso a operar y puede contener descritos en la Tabla 2.

**<valor>**: Valor solo necesario en operaciones de escritura y puede ser de 0 a 32 bytes.

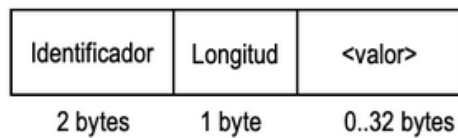
Tabla 1.

| Valor | Acción        |
|-------|---------------|
| 0000  | -             |
| 0001  | Lectura (R)   |
| 0010  | Escritura (W) |
| 0011  | -             |
| 0100  | Acceso (L)    |
| 0101  | Activo (K)    |
| 0110  | -             |
| 0111  | -             |
| 1000  | -             |
| 1001  | -             |
| 1010  | -             |
| 1011  | -             |
| 1100  | -             |
| 1101  | -             |
| 1110  | -             |
| 1111  | -             |

Tabla 2.

| Valor | Recurso      |
|-------|--------------|
| 0000  | -            |
| 0001  | LED (L)      |
| 0010  | ADC (A)      |
| 0011  | PWM (P)      |
| 0100  | -            |
| 0101  | -            |
| 0110  | -            |
| 0111  | -            |
| 1000  | -            |
| 1001  | -            |
| 1010  | -            |
| 1011  | -            |
| 1100  | -            |
| 1101  | -            |
| 1110  | -            |
| 1111  | Servidor (S) |

Cuando la petición por parte del servidor se pudo establecer de manera correcta, es cuando el programa del esp32 regresa un ACK junto con el valor asignado, el cual recibe el siguiente formato:



**<Identificador>**: Es un campo que funciona inicio del frame y corresponde al valor 0x35 0x01. (complemento de 0xCA 0xFE)

**<Longitud>**: Un campo que contiene la longitud en bytes del resto de la trama que es <valor> si es que la trama lo contiene y puede ser de 0 a 32 bytes.

**<valor>**: Valor solo necesario en operaciones de escritura y puede ser de 0 a 32 bytes.

Para el caso de **NACK**, la longitud deberá ser 0xFF con el campo <valor> vacío (0 bytes).

## Desarrollo:

La base del proyecto es muy parecida al proyecto de trama en ASCII, se utilizan las mismas variables globales y modificaciones de estas, entre las cuales las más importantes se encuentran

```
typedef enum{
    action_none = 0x00,
    read = 0x01,
    write = 0x02,
    access = 0x04, //login
    keep_alive = 0x05,
}action_t;
```

Se asigna a un enum el tipo de acción que se requiere hacer con dicho recurso de la aplicación.

```
typedef enum{
    led = 0x01,
    adc=0x02,
    pwm=0x03,
    server = 0x0f,
}resource_t;
```

Los recursos son los mismos, pero ahora estas litados con un índice que se representa de la siguiente manera, por un numero de 4 bits, por lo que deja un margen muy bien para una mayor cantidad de recursos.

La estructura para recibir las peticiones desde el servidor recibe cambios, porque ahora es necesario modificar el tipo de variable, así como los bits que cada variable estaría recibiendo ya que la trama tiene la siguiente estructura:

Por lo que la estructura recibe la siguiente modificación:

```
typedef struct{
    uint16_t id;
    uint8_t len;
    uint32_t user;
    action_t action:4;
    //recurso
    resource_t resource:4;
    uint8_t value[32];
}format_request_t;
```

Acción y recurso son un campo de 4 bits, por lo que usamos la asignación de bits para estas variables, para solo abarcar esos y entre los 2 solo se abarca 1 byte de tamaño, el valor es un campo de 32 bytes por lo que se hace uso de un arreglo de 32 espacios, cada uno es de 1 byte de tamaño.

Se modifica la estructura para enviar la trama por el tcp, en donde **op\_type** es el tipo de operación en las que se tienen:

```
typedef enum{
    OP_LOGIN,      // L:S
    OP_KEEPALIVE,  // K:S
    OP_ACK,        // ACK
    OP_NACK,       // NACK

}op_type_t;
```

Y se tiene el siguiente miembro el cual es el ya explicado en donde se ingresan los datos para enviar los datos, en donde lo usado para enviar seria **len** en el cual está el tamaño en bytes de del resto de la trama y tenemos **value** en donde va el dato a enviar, en contestación sobre el proceso hecho, si la operación resulto en un **ACK** en este campo no va nada en el caso de un **NACK**

```
typedef struct{
    op_type_t op_type;
    format_request_t *format_req_send;
}send_info_t;
```

## Archivo tcp\_lib.c

La función de **keep\_alive** se actualizo, dentro de esta función se construye la trama binaria a enviar hacia el servidor,

La parte del inicio es la importante. Tenemos un payload de 8 bytes ya que son todo el necesario para poder mandar un keep alive, en donde utilizamos **htnos (16 bits)** y **htnol (32 bits)** para convertir los bits que en del orden de la arquitectura de la computadora a la arquitectura de red que es **big-indian** antes de enviar, debemos de acomodar los bytes para que el servidor pueda recibir los datos de manera correcta.

En la variable keep hacemos un recorrido ya que el recurso y la acción compartir 1 byte, primero asignamos que queremos hacer un keep alive al servidor, y cada operación es un campo de 4 bits que se declararon en la estructura.

```

void keep_alive_task(void *params)
{
    while(1)
    {
        uint8_t payload[8];
        int offset = 0;

        uint16_t id = htons(HEADER);
        uint32_t user_keep = htonl(user);
        uint8_t keep = (keep_alive << 4) | server;

        memcpy(payload+offset, &id, 2); offset += 2;
        payload[offset] = 5; offset++;
        memcpy(payload+offset, &user_keep, 4); offset += 4;
        memcpy(payload+offset, &keep, 1); offset++;

        int err = send(tcp_client.sock, payload, offset, 0);
    }
}

```

## Funcion send\_message()

Cuando se requiere mandar información por ejemplo el **ACK o en NACK** se hace uso de la función que hace uso de la estructura **send\_info** la cual tiene la información como el valor a enviar, la longitud de este frame, e igual si es **ACK o un NACK** en donde se mantiene el mismo flujo, pero ahora se tiene un buffer de 40 bytes que es la cantidad máxima de bytes que se pueden enviar para este prototipo se necesitan 2 para el header **ACK** 1 que indica el tamaño de la trama empezando en el siguiente byte, y después el valor, que representa el estado actual que está el recurso que solicito el servidor escribir o leer.

```

esp_err_t send_message(){

    //lo maximo que puede enviar son 40 bytes estos puede variar
    uint8_t buffer[40];
    int offset= 0;
    int len;

    char hex_buf[128] = {0};
    int pos = 0;

    switch(send_info.op_type){

        case OP_LOGIN : {
            uint16_t id = htons(HEADER);
            memcpy(buffer + offset, &id, 2); offset += 2;
            buffer[offset] = 5; offset++;
            uint32_t user_htn = htonl(user);
            memcpy(buffer + offset, &user_htn, 4); offset += 4;
            uint8_t login = (access_esp << 4) | server;
            buffer[offset] = login; offset++;
        }break;
        case OP_ACK: {

            uint16_t id = htons(ACK);
            memcpy(buffer + offset, &id, 2); offset+=2;
            uint8_t payload_len = send_info.format_request.len;
            memcpy(buffer + offset, &payload_len, 1); offset +=1;
            if(payload_len > 32){
                return ESP_FAIL;
            }
            if(payload_len > 0){
                memcpy(buffer + offset, send_info.format_request.value, payload_len);
                offset += payload_len;
            }

        }break;
        case OP_NACK :{

            uint16_t id = htons(ACK);
            memcpy(buffer + offset, &id, 2); offset += 2;
            buffer[offset] = 0xFF; offset++;
        }break;

        default: {
            ESP_LOGI(TAG, "debugger");
        }break;
    }
    int sent = send(tcp_client.sock, buffer, offset,0);
}

```

## Funcion recv\_task:

La tarea encargada de recibir los datos que llegan del servidor se vio modificada para estar a la par del nuevo formato,

Pero debemos de verificar que es lo que llego, primero verificamos si el **recv** llega con un valor menor que 0 indica que ocurrió un error en la recepción de los bytes de la comunicación, por lo que cerramos para poder verificar de donde puede ocasionar el error ya que este tipo de errores idealmente no deben de pasar en la comunicación.

```
void recv_task(void *params){
    uint8_t rx_buffer[40];
    uint8_t *buffer_tmp;

    while(1){
        char hex_buf[128] = {0};
        int pos = 0;
        int len = recv(tcp_client.sock, rx_buffer, sizeof(rx_buffer)-1, 0);
        if(len < 0){
            if(errno == EAGAIN || errno == EWOULDBLOCK){
                vTaskDelay(pdMS_TO_TICKS(50));
                continue;
            }
            char err[80];
            snprintf(err, sizeof(err), "\r\nTCP_LIB: recv fallo errno: %d\r\n", errno);
            uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
            uart_write_bytes(global_uart.NUM_PORT, err, strlen(err));
            uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));

            tcp_client.connected = 0;
            tcp_client.logged_in = 0;
            if(tcp_client.sock >= 0){
                close(tcp_client.sock);
                tcp_client.sock = -1;
            }
            xEventGroupSetBits(g_tcp_event_group, TCP_DISCONNECTED);
            vTaskDelete(NULL); // esta tarea ya no tiene socket que leer
        }
    }
}
```

En el caso de que la respuesta será 0 indicia que la conexión se cerró por parte del servidor por lo que el socket actual ya no será valido para realizar las peticiones y requerimientos, por lo que igual cerramos conexiones del socket.

```
else if(len == 0){
    //conexion cerrada por el servidor
    char err[80];
    snprintf(err, sizeof(err), "\r\nTCP_LIB: el servidor cerro la conexion\r\n");
    uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
    uart_write_bytes(global_uart.NUM_PORT, err, strlen(err));
    uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
    tcp_client.connected = 0;
    tcp_client.logged_in = 0;
    close(tcp_client.sock);
    tcp_client.sock = -1;
    xEventGroupSetBits(g_tcp_event_group, TCP_DISCONNECTED);
    vTaskDelete(NULL); // esta tarea ya no tiene socket que leer
}
```



Otro caso indica que si se recibió un frame valido el con el cual vamos a poder actualizar los estados de los recursos.

Verificamos que tipo de frame es, si es un frame indicando una operación o es un **ACK** o un **NACK**.

La primera comparacion es que si es diferente a **0x3501** que es el identificador para cuando es un **ACK** o un **NACK**

Esto indica si es diferente es que viene un comando, el servidor quiere leer o escribir algo algun recurso, por lo que el mismo proceso que seguimos para enviar es algo muy parecido al recibir, vamos acomodando lo que recibimos en la estructura **format\_request** la cual después será enviado por una cola hacia la tarea **tpc\_proccess\_task**. En la cual se va a leer el contenido de esta estructura y se realizara las operaciones, esta tarea su único propósito es recibir y acomodar le bytes en un formato mucho más fácil de operar, mas no se toman decisiones.

```
uint8_t offset=0;
format_request_t *frame = pvPortMalloc(sizeof(format_request_t));

if (frame == NULL) {
    uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
    uart_write_bytes(global_uart.NUM_PORT, "\r\nsin memoria\r\n", 15);
    uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
    continue;
}

memcpy(&frame->id, &rx_buffer[offset], 2); offset +=2;
frame->id = ntohs(frame->id);

if(frame->id != 0x3501){
    memcpy(&frame->len,&rx_buffer[offset],1); offset++;

    memcpy(&frame->user, &rx_buffer[offset], 4); offset +=4;
    frame->user = ntohl(frame->user);

    uint8_t action_res = rx_buffer[offset]; offset++;
    frame->action = (action_res >> 4) & 0x0f;
    frame->resource = (action_res) & 0x0f;

    uint8_t value_len = (frame->len > 5) ? (frame->len - 5) : 0;
    if(value_len > 32) value_len = 32;
    if(value_len > 0){
        memcpy(frame->value, &rx_buffer[offset], value_len);
    }
}
if(frame->id == 0x3501 && rx_buffer[offset] == 0xff){
    //llego un nack
    memcpy(&frame->len,&rx_buffer[offset], 1); offset++;
}

if(frame->id == 0x3501 && rx_buffer[offset] != 0xff){
    memcpy(&frame->len,&rx_buffer[offset], 1); offset++;
}
//encolar
if(xQueueSend(tcp_rx_queue, &frame, 0) != pdTRUE){
    vPortFree(frame);
}
```

## Tarea tcp\_procces\_task

Es la tarea principal del programa ya que es donde se lee el contenido del frame recibido y se realiza dicha operación.

El frame más fácil de recibir es un **ACK** o **NACK** ya que solo contiene un dicho encabezado indicando que el frame se entendió o no entendió del parte del servidor.

```
2 void tcp_process_task(void *params){
3
4     format_request_t *frame;
5
6     char buffer[80]; //para mostrar mensajes por UART
7     esp_err_t ret;
8
9     uint8_t frame_len;
10    int len;
11    while(1){
12
13        //recibira los datos por cola
14        if(xQueueReceive(tcp_rx_queue,&frame, portMAX_DELAY)){
15
16            // ACK: identificador 0x3501 y len != 0xFF
17            if(frame->id == ACK && frame->len != 0xFF){
18
19                if(login_pending) {
20                    xEventGroupSetBits(g_login_event_group, LOGIN_SUCCESS);
21                    login_pending = 0;
22                }
23
24                len = snprintf(buffer, sizeof(buffer), "\r\nservidor contesta: ACK\r\n");
25                uart_write_bytes(global_uart.NUM_PORT, UART_GREEN, strlen(UART_GREEN));
26                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
27                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
28                vPortFree(frame);
29                continue;
30            }
31
32            // NACK: identificador 0x3501 y len == 0xFF
33            else if(frame->id == ACK && frame->len == 0xFF){
34
35                if(login_pending) {
36                    xEventGroupSetBits(g_login_event_group, LOGIN_FAIL);
37                    login_pending = 0;
38                }
39
40                len = snprintf(buffer, sizeof(buffer), "\r\nservidor contesta: NACK\r\n");
41                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
42                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
43                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
44                vPortFree(frame);
45                continue;
46            }
47        }
48    }
49 }
```

En otro caso quiere decir que recibidos un frame que indica un comando, primero verificamos que el comando sea para nosotros, para nuestro usuario en otro caso desechamos el frame y se pone en espera de recibir el próximo dato en la cola:

```
//en otro caso recibimos un comando del servidor (trama CAFE)
else if(frame->id == HEADER){
    frame_len = 0;

    //si llego ahora debemos de ver uqe onda

    //verificamos que sea para nosotros
    if(frame->user != user){
        len = snprintf(buffer, sizeof(buffer), "\r\npeticion no para este usuario\r\n");
        uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
        uart_write_bytes(global_uart.NUM_PORT, buffer, len);
        uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
        free(frame);
        continue;
    }
}
```

Ahora si es para nosotros lo que sigue en el frame es el recurso y la acción, si es lectura todos los recursos están disponibles para su lectura, para los datos de adc como el pwm es necesario aplicar la **función htons** para que la información se vea reflejada de manera correcta del lado del servidor, transforma el orden de bytes de la arquitectura de la pc en arquitectura de red necesaria en **big indician** de ahí en fuera las funciones son prácticamente iguales, para traer los valores de adc y de pwm.

```
//ahora necesitamos ver que servicio es lo necesario
if(frame->action == read_esp){
    switch(frame->resource){
        case led :{
            //el estado esta 1 o 0, que solo abarca 1 byte
            memcpy(send_info.format_request.value, &led_state, 1);
            send_info.format_request.len=1;//solo usamos 1 byte para el led
            send_info.op_type = OP_ACK; //operacion ACK
            ret = send_message();
            if(ret != ESP_OK){
                len = snprintf(buffer, sizeof(buffer), "\r\nERRO AL SER EL ENVIO DEL FRAME\r\n");
                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
            }
        }break;

        case adc:{
            //adc puede ser un valor de 16 bits
            uint16_t adc_state = read_adc(ADC_CHANNEL);
            uint16_t adc_net = htons(adc_state);
            memcpy(send_info.format_request.value, &adc_net, 2); //en este caso usamos 2 bytes
            send_info.format_request.len = 2;
            send_info.op_type = OP_ACK;
            ret = send_message();

            if(ret != ESP_OK){
                len = snprintf(buffer, sizeof(buffer), "\r\nERRO AL SER EL ENVIO DEL FRAME\r\n");
                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
            }
        }break;

        case pwm : {
            uint16_t duty = pwm_get_duty();
            uint8_t pct = (uint8_t)((duty * 100) / PWM_MAX);
            send_info.format_request.value[0] = pct;
            send_info.format_request.len = 1;
            send_info.op_type = OP_ACK;
            ret = send_message();

            if(ret != ESP_OK){
                len = snprintf(buffer, sizeof(buffer), "\r\nERRO AL SER EL ENVIO DEL FRAME\r\n");
                uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
                uart_write_bytes(global_uart.NUM_PORT, buffer, len);
                uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));
            }
        }break;

        default: {
            send_info.op_type = OP_NACK;
            ret = send_message();
        } break;
    }
}
```

Ahora si la **acción** es escribir, al único recurso que no se puede escribir es al ADC que viene asociado a un potenciómetro el cual es un dispositivo mecánico, por lo que por software no se puede alterar el valor del pwm solo puede ser leído.

Para el valor a escribí del led es sencillo ya que solo puede ser 1 o 0, en el primero byte del valor, por lo que es directo, la operacion con el pwm es necesario convertir el porcentaje que el servidor desea en el pwm en su representacion real de ciclo de trabajo del pwm, se realiza una operación para poder obtener ese valor.

Cuando se escrib el valor dentro del led o del pwm entonces se contesta con un **ACK** y el valor que se escribió al servidor indicando que la operación se realizó.

```

else if(frame->action == write_esp ){
    switch(frame->resource){

        case led:{
            //quiere escribir
            led_state = frame->value[0];
            gpio_set_level(OUTPUT_PIN, led_state);

            memcpy(send_info.format_request.value, &led_state, 1);
            send_info.format_request.len = 1;
            send_info.format_request.value[0] =led_state ;
            send_info.op_type = OP_ACK;
            //conestamos a la peticion
            ret = send_message();
        }break;

        case pwm : {
            uint8_t pct = frame->value[0];
            if(pct > 100) pct = 100;
            uint16_t duty = (pct * PWM_MAX) / 100;
            pwm_set_duty(duty);
            //devolver el ACK
            uint16_t duty = pwm_get_duty();
            uint8_t pct = (uint8_t)((duty * 100) / PWM_MAX);
            send_info.format_request.value[0] = pct;
            send_info.format_request.len = 1;
            send_info.op_type = OP_ACK;
            ret = send_message();
        }break;
        case adc: {
            len = snprintf(buffer, sizeof(buffer), "\r\noperacion con ADC incorrecta\r\n");
            uart_write_bytes(global_uart.NUM_PORT, UART_RED, strlen(UART_RED));
            uart_write_bytes(global_uart.NUM_PORT, buffer, len);
            uart_write_bytes(global_uart.NUM_PORT, UART_RESET, strlen(UART_RESET));

            //enviamos un NACK
            send_info.op_type = OP_NACK;
            ret = send_message();
        }break;

        default: {
            send_info.op_type = OP_NACK;
            ret = send_message();
        } break;
    }
}
}

```

Ejercicio a prueba:

Haciendo uso de un servidor en python en local, con la estructura igual a lo que esperaríamos el servidor en línea.

Solicitamos escribir un 1 al led, el led recibe este comando, prende el led y regresa el estado actual del led:

```

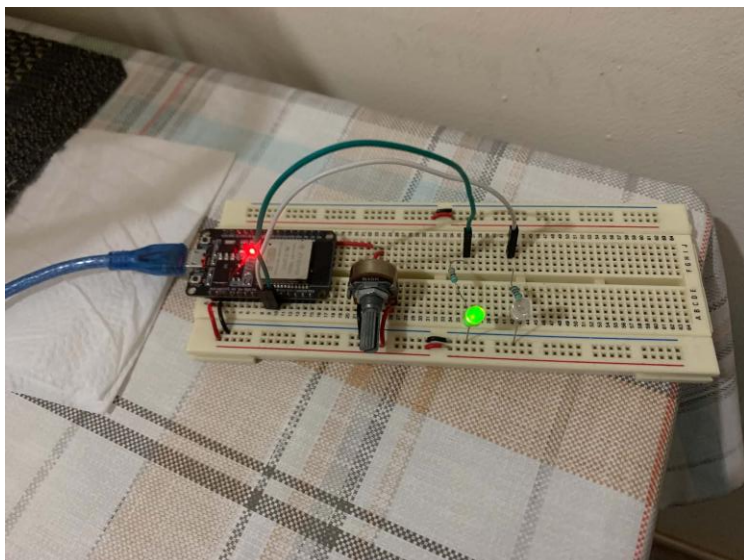
5
Cliente: ('192.168.1.93', 54965) user=0x001377D7 logueado
Valor (0 o 1): 1[21:58:34] DEBUG | [['192.168.1.93', 54965]] RX raw: CA FE 05 00 13
[21:58:34] INFO | [['192.168.1.93', 54965]] RX + user=0x001377D7 acción=keepalive(0x
io)
[21:58:34] INFO | [['192.168.1.93', 54965]] KEEP-ALIVE de 0x001377D7
[21:58:34] DEBUG | [['192.168.1.93', 54965]] TX + 35 01 00

→ Enviando: Escribir LED 01
[21:58:34] DEBUG | [['192.168.1.93', 54965]] TX + CA FE 06 00 13 77 D7 21 01

MENÚ DE OPERACIONES
1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [21:58:34] DEBUG | [['192.168.1.93', 54965]] RX raw: 35 01 01 01
[21:58:34] INFO | [['192.168.1.93', 54965]] ESP32 respondió ACK → LED = ON (1)
[21:58:34] DEBUG | [['192.168.1.93', 54965]] RX raw: CA FE 05 00 13 77 D7 5F

```



Ahora si elegimos leer el timer de primero regresara un 0% ya que en el código esta inicializado de dicha forma:

```

3
Cliente: ('192.168.1.93', 54965) user=0x001377D7 logueado
→ Enviando: Leer PWM (sin valor)
[22:01:39] DEBUG | [['192.168.1.93', 54965]] TX → CA FE 05 00 13 77 D7 13

MENÚ DE OPERACIONES
1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [22:01:39] DEBUG | [['192.168.1.93', 54965]] RX raw: 35 01 01 00
[22:01:39] INFO | [['192.168.1.93', 54965]] ESP32 respondió ACK → PWM = 0%
[22:01:44] DEBUG | [['192.168.1.93', 54965]] RX raw: CA FE 05 00 13 77 D7 5F

```

Ahora escogemos escribir al PWM un 50% de ciclo de trabajo:

```

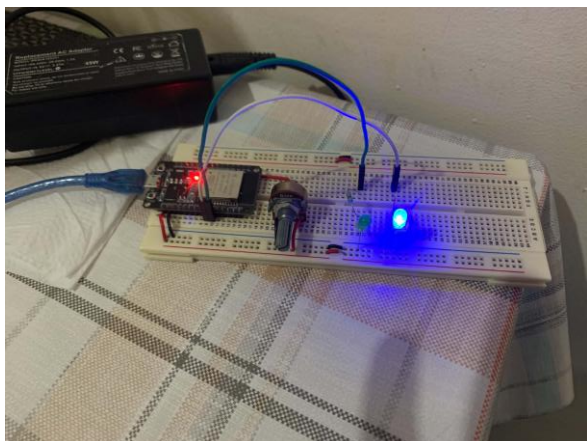
6
Cliente: ('192.168.1.93', 52822) user=0x001377D7 logueado
Valor (0-100 (%)): 50
→ Enviando: Escribir PWM (%) 32
[22:25:27] DEBUG | [['192.168.1.93', 52822]] TX → CA FE 06 00 13 77 D7 23 32

MENÚ DE OPERACIONES
1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [22:25:27] DEBUG | [['192.168.1.93', 52822]] RX raw: 35 01 01 00
[22:25:27] INFO | [['192.168.1.93', 52822]] ESP32 respondió ACK → 0x00 = 0

```

Podemos ver que en el recurso se observa un led prendido no a su máxima capacidad ya que solo está al 50%



Si leemos el PWM obtendremos el valor actual que tiene:

```

3
  Cliente: ('192.168.1.93', 52822) user=0x001377D7  logueado
  → Enviando: Leer PWM (sin valor)
[22:28:16] DEBUG | [('192.168.1.93', 52822)] TX → CA FE 05 00 13 77 D7 13

MENÚ DE OPERACIONES

1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [22:28:16] DEBUG | [('192.168.1.93', 52822)] RX raw: 35 01 01 31
[22:28:16] INFO | [('192.168.1.93', 52822)] ESP32 respondió ACK → PWM = 49%
[22:28:16] DEBUG | [('192.168.1.93', 52822)] TX → CA FE 05 00 13 77 D7 12
  
```

Ahora escogemos leer el ADC:

```

2
  Cliente: ('192.168.1.93', 52822) user=0x001377D7  logueado
  → Enviando: Leer ADC (sin valor)
[22:27:44] DEBUG | [('192.168.1.93', 52822)] TX → CA FE 05 00 13 77 D7 12

MENÚ DE OPERACIONES

1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [22:27:44] DEBUG | [('192.168.1.93', 52822)] RX raw: 35 01 02 0F FF
[22:27:44] INFO | [('192.168.1.93', 52822)] ESP32 respondió ACK → ADC = 4095 (3.3 V)
  
```

Actualmente se encuentra en el máximo, pero si bajamos el ADC este valor debería de bajar igual:



```
2
Cliente: ('192.168.1.93', 52822) user=0x001377D7 logueado
→ Enviando: Leer ADC (sin valor)
[22:29:09] DEBUG | [['192.168.1.93', 52822]] TX → CA FE 05 00 13 77 D7 12

MENÚ DE OPERACIONES
1. Leer LED
2. Leer ADC
3. Leer PWM
4. Leer timer RESET
5. Escribir LED
6. Escribir PWM (%)
7. Programar RESET
8. Cancelar RESET
9. Listar clientes
10. Salir

Opción: [22:29:09] DEBUG | [['192.168.1.93', 52822]] RX raw: 35 01 02 07 43
[22:29:09] INFO | [['192.168.1.93', 52822]] ESP32 respondió ACK → ADC = 1859 (1.498 V)
```

## Conclusiones y dificultades:

Para el desarrollo de este prototipo de IoT con trama binaria nos dejó ver que este tipo de trama entre 2 dispositivos ahorra demasiado ancho de banda en donde para ASCII los 40 bytes no alcanzaría para construir la trama completa con la trama binaria basta y sobra para poder aun adjuntar muchos más recursos y operaciones, incluso si se simplifica puede tener muchos más recursos, ya que las acciones por lo general son más pocas, con 3 bits es suficiente para abarcar la mayoría de las acciones dejando 5 bits disponibles para poder tener más recursos.

Como dificultad el cómo el sistema iba a mandar los datos, en esta parte es en donde más me perdí, el cómo los iba a recibir, en que formato u orden venían y en cómo se deberían de ir, fue en donde más tarde en pensar cual sería la forma más optimiza más sencilla de recibir y acomodar los diferentes bytes de la trama en donde correspondieran, una vez que solucione ese problema el demás proceso era casi igual a cuando se manejaba la trama en ASCII solo que hay que tener un poco más de cuidado con los tipos de datos y los casting necesario para que lo que se reciba y lo que se envia sea lo correcto, el estado correcto de los recursos.